

Building a new dashboard — team guide

Files: boilerplate.html · design-tokens.css · widgets.css · widget-scripts.js Updated: August 2026

boilerplate.html

design-tokens.css

widgets.css

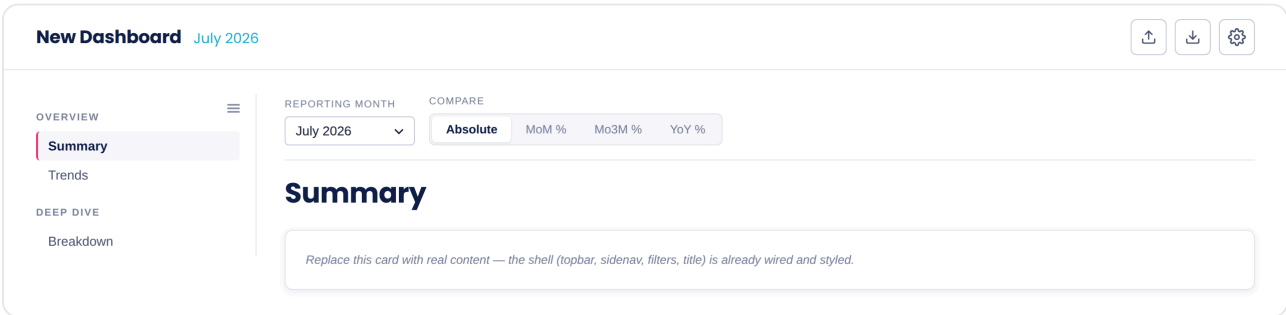
widget-scripts.js

You're not meant to hand-edit these files. They're context you hand to Claude, not a codebase you open yourself. Attach the ones relevant to your task, describe what you're building, and Claude does the copying, wiring, and adapting. Everything below is written as what-to-attach-and-what-to-say — the file/class names still show up, because that's the vocabulary that gets Claude to the right part of the source files quickly, not because you're expected to go find them yourself.

Four files: **boilerplate.html** (the page shell — topbar, sidenav, filters, page titles, section switching, already wired and working), **design-tokens.css** (embedded inside the boilerplate already, so attaching that one file covers it), **widgets.css** (an opt-in catalog of fancier per-report CSS components — chart modals, driver tables, pyramid grids, and so on), and **widget-scripts.js** (the same idea as widgets.css but for widgets substantial enough to need real JS, not just CSS — a PPTX export engine, Excel/CSV ingestion helpers, and an in-browser AI data assistant).

The rule

Start every new dashboard by attaching boilerplate.html and building from it. Don't start from a blank chat, and don't ask Claude to "match the style of Product Pulse" from memory — attach the actual file. A description gets Claude something *similar*; the file gets it something *identical*. This is the single most important habit in this whole guide — everything else below assumes you're already doing this.



boilerplate.html, unmodified — topbar, sidenav, filters, and the placeholder content card, all already wired and styled.

Set this up once, not per chat

Don't attach `boilerplate.html` fresh every time and only reach for `widgets.css` or `widget-scripts.js` once a task turns out to need them. Preload all four files from the start:

- **Best option — a Claude Project.** Create a project for this dashboard system, add `boilerplate.html`, `design-tokens.css`, `widgets.css`, and `widget-scripts.js` to its project knowledge once, and every new chat inside that project already has all four without you re-attaching anything. It also means Claude can proactively flag "there's already a driver table widget for this" instead of you having to remember the catalog exists and ask for it by name.
- **Without a Project:** attach all four files at the start of a chat even if you think you only need `boilerplate.html` for that particular task. It costs nothing to have the widget catalogs sitting there, and it means Claude can flag a relevant widget partway through a conversation instead of only when explicitly asked.

Either way, the goal is the same: Claude should have the full picture before you start describing what you want, not receive it piecemeal as the conversation reveals what's needed.

Starting a new dashboard

Attach `boilerplate.html` and say something like:

"Here's our dashboard boilerplate. Build me a [X] dashboard using this exact structure — keep the topbar, sidenav, and filters as-is, add sections for [Y, Z], and use the existing button/badge/pill/segmented classes already defined in the file rather than inventing new styles."

The more specific you are about what sections/filters you need, the less Claude has to guess — but you never need to re-specify the visual language (colors, spacing, shadows, component look), because it's already sitting in the file you attached.

Bringing an existing dashboard onto the shared system

This is a different job from starting fresh: you have a dashboard that predates the template — its own bespoke CSS, its own topbar, maybe its own filter logic — and you want it to look and feel like the rest of the Pulse family without rebuilding it from scratch or breaking what already works.

Attach the existing dashboard alongside `boilerplate.html` and be explicit about the split between "restyle" and "rebuild":

"Here's an existing dashboard that wasn't built from our template, and here's the current boilerplate. Bring its topbar, sidenav, filters, and colors/fonts/shadows in line with the boilerplate's shared design system — but don't touch its data logic, its filters, or its chart code. Flag anything that conflicts (a custom color, a different grid system, a class name that collides with one of ours) instead of silently overriding it."

A few things worth doing on a retrofit like this that you wouldn't need to think about when starting fresh:

- **Do the shell/token swap first, on its own**, before asking for any widgets from `widgets.css`. Confirm the dashboard still works — same data, same filters, same charts — before layering anything else on top. Changing the visuals and the logic in the same pass makes it much harder to tell which change broke what if something did.
- **Ask Claude to check for collisions before merging**, not after. An existing dashboard built independently may already have its own `.pill`, `.chart-card`, or `--navy`-equivalent token under a different name — ask Claude to flag those rather than assume the merge is clean.
- **Treat the old dashboard's control types as inherited, not required**. Carrying over the SQL/state logic behind a filter (which column it filters, single- vs. multi-select, how it narrows other filters) isn't the same as carrying over its widget — pills vs. segmented vs. dropdown vs. checkboxes. The old choice may just be whatever came to hand in the source file rather than a deliberate one, and the boilerplate may already have its own idiom for that kind of control. Ask Claude to flag which widget each filter/control used to be and which one you want it to become, rather than defaulting to matching the source file's markup.
- **Test against the dashboard's real embedded data, not just visually**. Every Pulse dashboard bakes its dataset in as an embedded CSV string, parsed client-side on load (typically with PapaParse) — there's no server and no external build step, it's all self-contained inside the file Claude produces. A class name or selector that looks fine in a static render can still break the JS that reads the parsed rows, and a visual pass alone won't catch that. (The `widget-scripts.js` CSV helpers cover the upload-a-new-file flow, not this baked-in-at-build-time one — if a dashboard needs help with its own embedded-CSV parsing logic, that's dashboard-specific code to review directly, not something cataloged here yet.)
- **Expect to go back and forth**. Unlike a from-scratch build, a retrofit usually surfaces a few real conflicts — a color clash, a layout assumption that doesn't hold — that need a decision from you, not just an instruction to Claude. Budget for a round of "here's what I found, which do you want" before it's done.

What's fair game to ask for vs. what to push back on

Everything in the left column below is a normal, reasonable ask. Everything in the right column is the shared system that every Pulse dashboard leans on — if Claude proposes changing one of those without you asking for it (or if you're tempted to ask "can you just tweak the shadow on this one card"), that's usually a sign to redirect it back to the existing pattern instead.

Fair game to ask Claude for	Push back if Claude touches this unasked
Sidenav section labels & count	Topbar layout, icon buttons, and the brand-mark title component
Filter controls (which dropdowns/segments you need)	The sidebar's collapse/drawer behaviour
Page titles and section content	Color tokens, fonts, shadows, radius
Number/type of sections	The filters bar's spacing and structure
New settings rows under the gear icon	The mobile breakpoints and their rules
A rename of the theme's localStorage key per dashboard	The light/dark toggle itself and how it's wired

If a new dashboard needs a filter type not shown in the boilerplate (e.g. a brand pill-picker), that's a completely normal ask — just tell Claude to build it from the existing brand-badge/segmented/select-wrap classes rather than inventing new CSS. Same idea for settings: ask for new rows under the existing Appearance row rather than a second popover.

Adding a new section

Just describe the section — you don't need to explain the mechanics:

"Add a new section called Retention to the sidenav. Give it a page title and I'll fill in the content after — follow the same pattern the other sections use."

Claude already has everything it needs once `boilerplate.html` is attached: the section-switching script in the file handles showing and hiding based on matching a sidenav button to its section, so there's no new logic to write and nothing for you to specify beyond what the section is called and what goes in it.

Pulling in a widget (widgets.css)

The boilerplate shell stays minimal on purpose — the fancier components the source dashboards built for their own report content (chart modals, driver tables & waterfalls, the pyramid/metric grid, definition/summary tables, threshold chips, brand filtering, the multi-file upload flow, scorecards, a Key Changes tab, an AI assistant panel shell, a data dictionary page, a brand-split overlay picker, and a handful of smaller utilities) live in **widgets.css** instead of the boilerplate, so a plain new dashboard doesn't inherit components it doesn't need.

You don't need to open the file or copy any CSS yourself — attach `widgets.css` alongside `boilerplate.html` and name what you want:

"Attach widgets.css too. Use the boilerplate as the base, and pull in the driver table + waterfall widget for the Drivers section, plus the threshold chips on the summary cards."

"Use the Key Changes tab widget from widgets.css, but adapt it to my three verticals — Sports, Casino, and Bingo — instead of the two the source report has."

"Add a Definitions page using the data dictionary widget from widgets.css. Here's our metric list and what each one means — build the tables from this, don't reuse Commercial Pulse's simpler def-table."

"Add a vertical filter to the filters bar — a plain select, Sports vs. Casino vs. Bingo — following the same pattern as the existing Reporting Month dropdown."

The widget names to reach for: *Chart card & modal, Driver table & waterfall, Metric/pyramid grid, Definition/HV/summary tables, Data dictionary page, Threshold chips, Brand filtering, Brand-split overlay picker, Multi-chart comparison panel, Missing-data controls, Upload/dropzone flow, Key changes tab, AI data assistant (panel shell), Small utilities*. You don't need the exact wording — "the pyramid grid one" or "the thing that shows a trend chart when you click a figure" is enough for Claude to find the right section.

Ask Claude to flag whether the widget you want is verbatim-identical across the source dashboards or was only built for one of them — the latter is still safe to use, just worth a second look to confirm it fits your data shape, since it's only been proven out on one report so far. Everything in `widgets.css` is built from the same tokens as `design-tokens.css`, so nothing pulled in will clash visually with the boilerplate.

If nothing in the catalog covers what you need, describe it using the same constraints rather than asking for something open-ended — "use our navy/cyan palette and the same shadow style as the rest of the dashboard" — and consider asking Claude to add the result back into `widgets.css` once it's proven out, so the next dashboard doesn't have to rebuild it from scratch.

Pulling in a script-heavy widget (widget-scripts.js)

Seven widgets needed enough real JS behaviour that they outgrew `widgets.css`'s CSS-only format: a PowerPoint export that produces a genuinely *editable* native chart (not a pasted image), a set of Excel/CSV ingestion helpers robust to however a real analyst's export actually formats dates and percentages, an in-browser "ask the dashboard a question" AI assistant, the lag math behind a MoM/Mo3M/YoY compare toggle, a brand-split overlay chart builder, the render pattern behind a data dictionary page, and a from-scratch "export this chart as a standalone HTML file" feature. These live in **widget-scripts.js**, attached the same way:

"Attach widget-scripts.js and widgets.css too. Add the native PPTX export to the chart modal, and the AI assistant — our brands are [...], our key metrics are [...]."

"Wire up the Compare toggle so MoM/Mo3M/YoY actually recompute the chart, using the lag-math widget from widget-scripts.js."

"Add brand-split overlay to the trend chart — let me pick which brands show as overlaid lines, using the widget-scripts.js pattern."

The PPTX export engine and the Excel/CSV helpers are fully generic — just ask for them, no extra detail needed. Same for the MoM/Mo3M/YoY lag math — it's five lines with no report-specific content in it at all. The AI assistant, the brand-split overlay chart, and the data dictionary page are different: their mechanics carry over unchanged, but the parts that touch your actual data — brand names, metric labels, date handling, your methodology text, your lookup of which numbers go where — are specific to the report they came from and have to be rewritten for yours. Give Claude your brand/metric vocabulary up front (or point it at your dataset) so it can do that rewrite in one pass rather than guessing and getting corrected repeatedly.

One more worth flagging on its own: the "export chart as raw HTML" widget wasn't found in any of the three source dashboards — none of them had actually built it, despite being on the original wishlist. It's written fresh rather than extracted, so treat it as a starting point to test against your own charts, not as something already proven the way the PPTX engine or the AI assistant are.

Visual reference

Representative samples of the most-used `widgets.css` and `widget-scripts.js` components, rendered with placeholder data — useful for pointing and saying "like this one" instead of describing a widget from scratch. Colours shown use the corrected Flutter brand palette now in `design-tokens.css`.

CHART CARD & MODAL



DRIVER TABLE & WATERFALL

DRIVER	IMPACT
Brand awareness	+2.1pp
Ease of use	-1.4pp
Value perception	+0.8pp
Net movement	+1.5pp

METRIC / PYRAMID GRID

	Sky Bet	Paddy Power	Bet365
HV+	Consideration 54%	Consideration 61%	Consideration 58%
HV	Consideration 47%	n/a	Consideration 52%

DEFINITION / SUMMARY TABLES

Metric	Definition
Consideration	% who would consider betting with this brand next
NPS	% promoters minus % detractors

THRESHOLD CHIPS & BRAND FILTERING

+2.1pp ▲

-1.4pp ▼

Sky Bet

Paddy Power

Betfair

tombola

BRAND-SPLIT OVERLAY PICKER

Sky Bet -

Paddy Power -

Betfair +

tombola +



DATA DICTIONARY PAGE

The dashboard is based around three North Star Metrics relating to: **Volume**, **Conversion** and **Value**.

Comparison references

LABEL	DEFINITION
MoM	Month on month (comparison to previous month)
Mo3M	Month on 3 month (comparison to 3 months earlier)
rm_events	Real-money bets/spins — the Conversion North Star Metric

MULTI-FILE UPLOAD / DROPZONE FLOW

Tracker data (.csv / .xlsx)

Drop file or click to browse

✓ Q3_tracker.xlsx — 1,842 rows loaded

REQUIRED

Metric map (.csv)

Drop file or click to browse

OPTIONAL

KEY CHANGES TAB

Statistically Significant Changes

Decreases

KEY ATTRIBUTES

Ease of use42% -5.1pp YoY

Notable Trends

Upward Trends

ADDITIONAL METRICS

Value perception

SCORECARD & SMALL UTILITIES

CONSIDERATION

54%

▲ 2.1pp

NPS

+18

▼ 1.4pp

AWARENESS

81%

— 0.0pp

AI DATA ASSISTANT

Data Assistant

Runs in your browser

Sky Bet's Consideration is

54%

vs Paddy Power?

Sample data shown for illustration only — ask Claude for the widget by name (see "Pulling in a widget" above) rather than describing this image back to it.